

Lab -ASP.NET - Creating a Container

To Begin with the Lab

Summary

In this lab, you connect an ASP.NET application to an existing Azure Storage Account using a **Storage Account Connection String**. You install the Azure Blob Storage SDK, build the application, and run it from the terminal to create a Blob Container programmatically. This approach allows applications to create and manage Blob Storage resources directly through code instead of using the Azure Portal.

Understanding

A **Storage Account Connection String** contains the information required for an application to authenticate and connect to an Azure Storage Account. Once the application connects, it uses the **BlobServiceClient** class to communicate with Azure Blob Storage. A

BlobContainerClient is then used to access or create a specific Blob Container.

The **CreateIfNotExistsAsync()** method first checks whether the container already exists. If it does not exist, Azure creates it automatically. If the container already exists, no error occurs, making the application safe to run multiple times. Using the .NET CLI commands (dotnet add package, dotnet build, and dotnet run) simplifies package installation, compilation, and execution directly from the terminal.

Example

Suppose you already have an Azure Storage Account named **mystorageaccount**.

Connection String:

DefaultEndpointsProtocol=https;

AccountName=mystorageaccount;

AccountKey=xxxxxxxxxxxxxxxxxxxx;

EndpointSuffix=core.windows.net

Container to Create:

images

Application Flow:

ASP.NET Application



Connection String



Azure Storage Account



BlobServiceClient



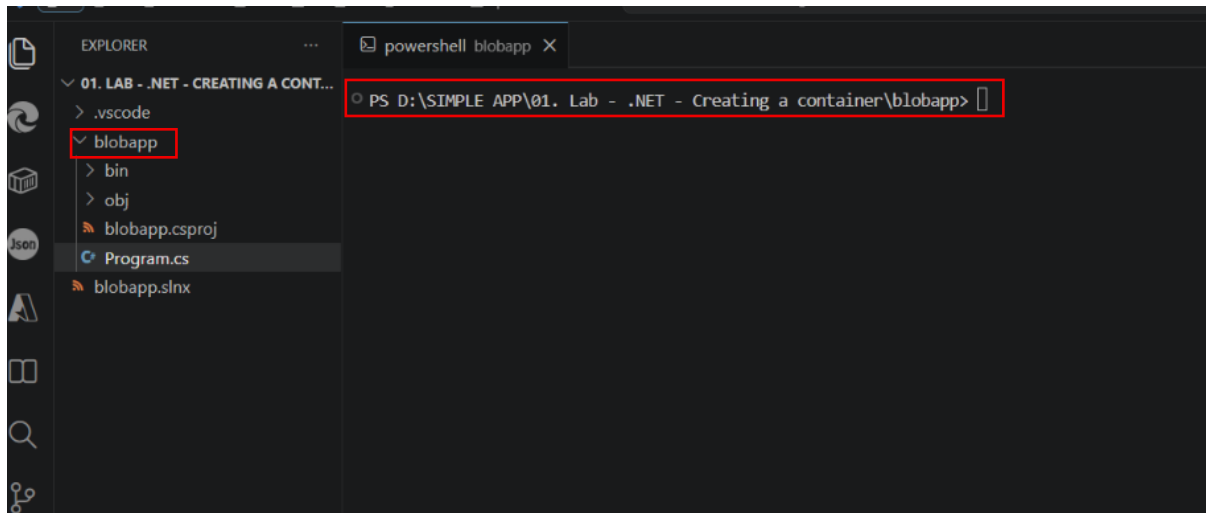
Blob Container (images)

When the application runs, it connects to the Storage Account and automatically creates the **images** container if it does not already exist.

Lab Steps

Step 1: Open the ASP.NET Project

- Open your ASP.NET project in Visual Studio or Visual Studio Code.
- Open the integrated terminal.
- Ensure you are in the project directory.

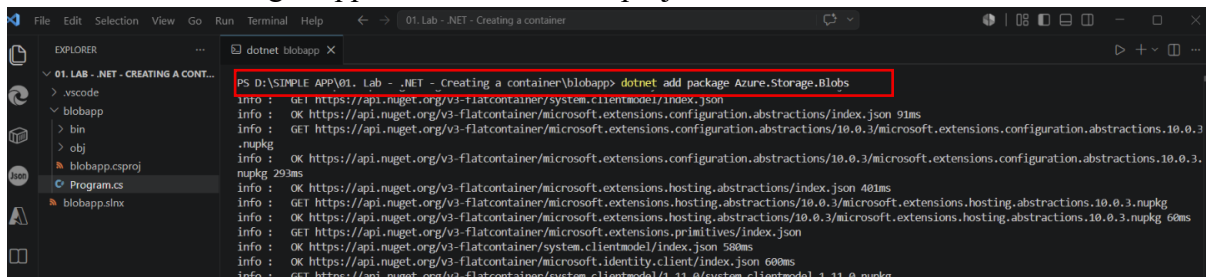


Step 2: Install the Azure Blob Storage Package

Run the following command:

```
dotnet add package Azure.Storage.Blobs
```

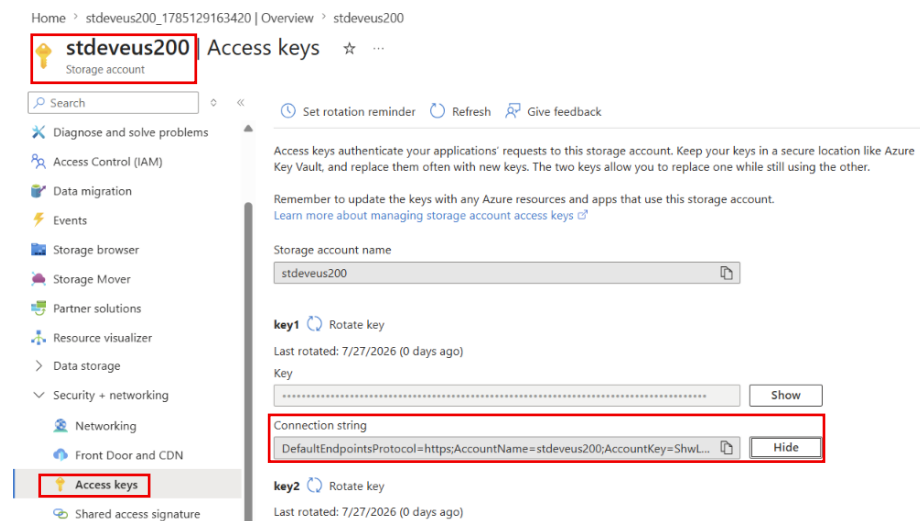
- Installs the Azure Blob Storage SDK.
- Adds Blob Storage support to the ASP.NET project.



Step 3: Add the Storage Account Connection String

- Go to exist **Storage** accounts.
- Navigate to **Access keys**.

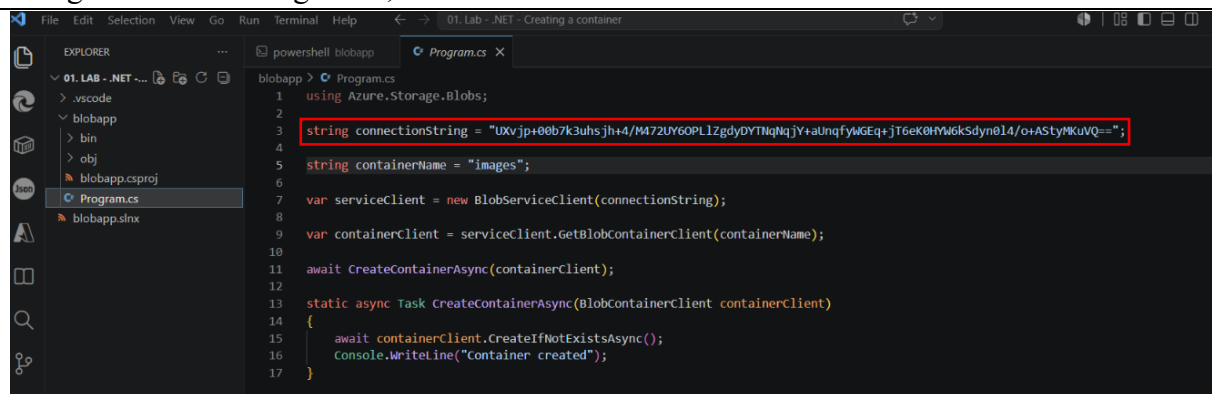
- Copy the Storage Account Connection String from the Azure Portal.



- Add the connection string to your application.

Example:

```
string connectionString = " ";
```

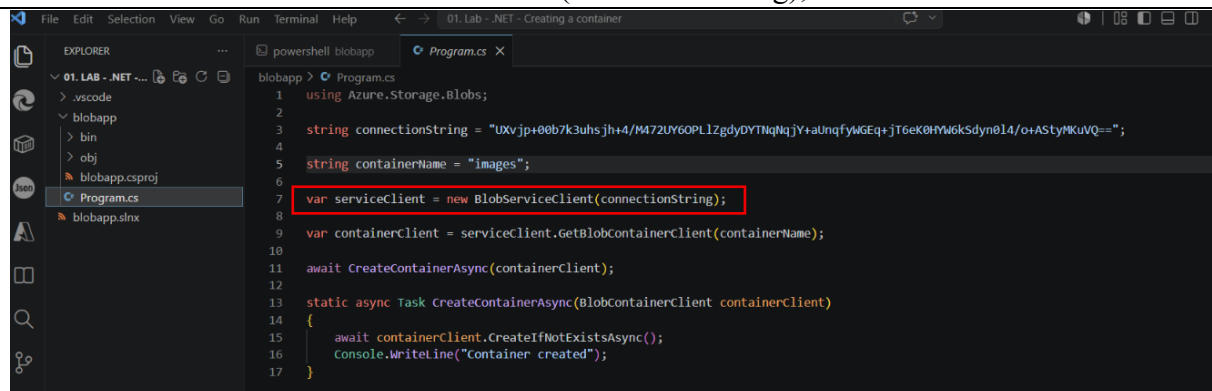


Step 4: Create the Blob Service Client

- Create a BlobServiceClient using the Storage Account Connection String.
- This client establishes the connection with Azure Blob Storage.

Example:

```
var serviceClient = new BlobServiceClient(connectionString);
```



Step 5: Create the Blob Container

- Create a BlobContainerClient.
- Example:

```
await CreateContainerAsync(containerClient);

static async Task CreateContainerAsync(BlobContainerClient containerClient)
{
    await containerClient.CreateIfNotExistsAsync();
    Console.WriteLine("Container created");
}
```

- Specify the container name.
- Example:

```
string containerName = "images";
```

- Use `CreateIfNotExistsAsync()` to create the container only if it does not already exist.

```
1 using Azure.Storage.Blobs;
2
3 string connectionString = "UXvjp+00b7k3uhsjh+4/MA72UY60PL1ZgdyDYTNqNqjY+aUnqfyWGEq+jTGeK0HYW6kSdyn014/o+ASTyMKuUQ==";
4
5 string containerName = "images";
6
7 var serviceClient = new BlobServiceClient(connectionString);
8
9 var containerClient = serviceClient.GetBlobContainerClient(containerName);
10
11 await CreateContainerAsync(containerClient);
12
13 static async Task CreateContainerAsync(BlobContainerClient containerClient)
14 {
15     await containerClient.CreateIfNotExistsAsync();
16     Console.WriteLine("Container created");
17 }
```

Step 6: Build the Application

Run the following command:

```
dotnet build
```

- Compiles the application.

```
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container\blobapp> dotnet build
Restore complete (0.5s)
blobapp net10.0 succeeded (5.4s) -> bin\Debug\net10.0\blobapp.dll

Build succeeded in 6.6s
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container\blobapp>
```

Step 7: Run the Application

Run the following command:

```
dotnet run
```

- Starts the ASP.NET application.
- Connects to Azure Storage.
- Creates the Blob Container.

Expected Output:

Container created successfully.

```
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container\blobapp> dotnet run
Container created
PS D:\SIMPLE APP\01. Lab - .NET - Creating a container\blobapp>
```

Step 8: Verify the Container

- Open the Azure Portal.
- Navigate to your Storage Account.
- Select **Data Storage** → **Containers**.
- Verify that the **images** container has been created successfully.

